

国产算力平台大模型推理引擎综述

张书豪¹, 廖小飞¹

¹ 大数据技术与系统国家地方联合工程研究中心

服务计算技术与系统教育部重点实验室

集群与网格计算湖北省重点实验室

华中科技大学计算机科学与技术学院

一、引言

设想这样一个场景：一支工程团队接到任务，要把公司正在使用的大模型推理服务从英伟达 GPU 迁移到一款国产加速芯片上。框架选定 vLLM，模型权重已就绪，芯片厂商也提供了驱动和工具链。看起来离“跑起来”只差最后一步——但真正动手之后，问题接踵而来。算子库缺少几个关键融合路径，图模式在动态形状下频繁回退，通信库版本与框架版本存在隐式耦合，量化路径在这块芯片上精度异常……这些问题单独看都不算致命，但它们同时出现、彼此关联，使原本“换一块卡”的工作迅速膨胀为一项需要跨越编译器、运行时、调度器和运维工具的系统工程。这不是个别团队的遭遇，而是当前国产算力推理引擎领域几乎所有参与者都在面对的共同处境。

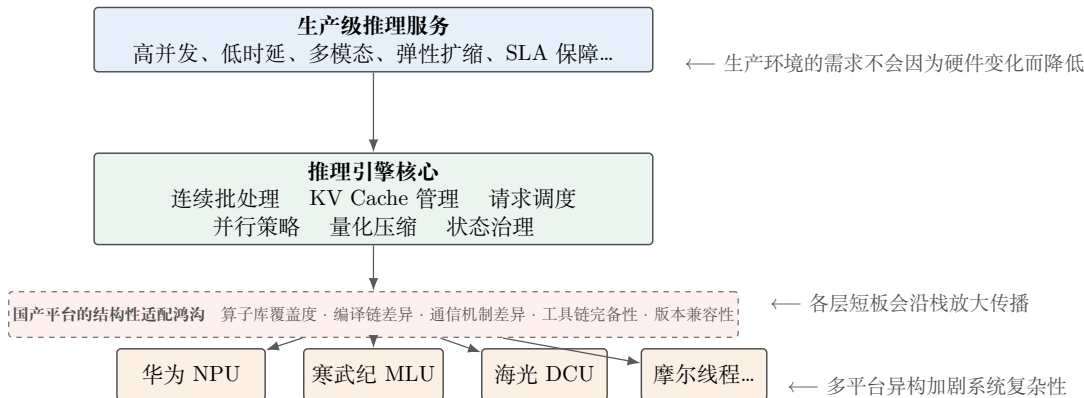


图 1: 国产算力平台大模型推理全栈挑战示意图：生产级服务需求与多平台异构硬件之间的结构性鸿沟

近两年，开源社区在推理引擎设计上积累了大量成果。vLLM 通过 PagedAttention 和连续批处理改写了大模型服务系统的内存与调度范式 [1, 2]；SGLang 把请求编排和结构化控制流提升为一等抽象 [3, 4]；Orca 与 Sarathi-Serve 确立了迭代级调度与分块预填充作为高吞吐服务的基础组织 [5, 6]；DistServe 与 Splitwise 将 prefill 和 decode 阶段解耦从一个优化技巧变成了整体吞吐设计的核心问题 [7, 8]；XGrammar 和 ElasticMM 则把推理引擎的边界推向了结构化输出和多模态并行 [9, 10]。这些工作在 CUDA 生态中已趋于成熟，但当它们进入国产芯片和本土部署环境后，新的挑战不再是“有没有这个功能”，而是“统一的框架抽象如何与不断变化的本土设备能力持续对齐”。

*受新一代人工智能国家科技重大专项资助（课题编号：2025ZD0123804）

通信作者：廖小飞，E-mail: xfliao@hust.edu.cn

从公开生态来看，三类技术路径已经清晰分化：有人选择在开源主干上增加后端适配，快速复用社区能力；有人选择由平台厂商把编译器、运行时和推理引擎打包为一体化方案，追求交付确定性；也有人选择自研引擎或混合架构，试图在兼容与自主之间找到平衡。三者的根本差异不在于用了什么优化技巧，而在于对“系统边界画在哪里、演进节奏由谁决定”这个问题给出了不同的回答。本文的目的不是为这些项目编写名录，而是提供一个分析骨架来理解它们背后的共性与分歧。这个骨架由“一个底座加三条主路径”构成：底座是统一抽象、统一指标和可观测证据链——它为比较提供共同语言；三条主路径分别是执行与通信、状态管理、压缩与成本协同——它们是区分不同系统真正做出何种取舍的维度。沿着这个骨架，读者可以判断一个系统究竟把主要复杂度压在了后端实现、平台运行时、服务状态机还是工具链控制面上，进而理解哪些能力应当沉淀为稳定接口、哪些状态必须纳入运行时统一管理、哪些优化适合以插件形式独立演进。

最后需要说明本文的证据使用方式。讨论建立在可交叉核验的公开材料之上：系统论文与技术报告用于厘清机制脉络；官方文档与开源仓库用于确认能力边界；社区部署记录用于补充真实工程中的摩擦点。对于仅见于单次演示或缺乏复现链路材料，不据此给出性能与成熟度结论。路线比较围绕可演进性、状态管理能力、复杂负载承载能力和交付确定性展开，不试图在缺少统一实验条件的前提下给出跨平台的绝对性能排序。

二、面向国产算力的大模型推理引擎生态

当一家企业决定在国产芯片上部署大模型推理服务时，它实际上拿到的是什么？在 CUDA 生态中，答案相对清楚：选一个推理框架，加载模型，配好并行策略，背后的算子库和通信库经过多年打磨，大多数情况下开箱即用。但在国产平台上，这个答案会迅速分叉。拿到的可能是厂商提供的一套端到端方案，也可能是一个开源框架的社区插件，还可能是一个试图统一多类硬件的中间抽象层——每种选择背后的调优深度、生态兼容性和演进节奏都不一样。生态为什么会呈现这样的格局？不是因为某个项目或厂商做了特别的选择，而是因为国产算力市场中存在两股互相拉扯的力量，它们共同塑造了整个生态的走向。

第一股力量是硬件厂商与开源社区之间的节奏差。举个具体的例子：当一个新的开源模型发布后，社区维护的 vLLM 或 SGLang 往往在一两周内就能跑通流程——但要在特定芯片上获得真正可用的性能，厂商的算子融合、图编译和低精度路径优化可能还需要几个月。厂商原生引擎（MindIE、MagicMind 等）的优势在于深度调优，但跟进新模型的速度拼不过社区；开源框架的优势在于模型覆盖广，但在单一硬件上的优化深度无法与厂商匹敌。这股张力催生了一种折中形态：以 vLLM 插件机制为代表，厂商在框架定义的抽象边界内实现后端算子，复用上游的调度、缓存和接口层，同时保留对关键算子的定制空间。当前几乎所有国产芯片——华为、寒武纪 MLU、摩尔线程 MUSA、沐曦 MetaX、昆仑芯、燧原 GCU——都已通过这种方式接入 vLLM 生态。这不是巧合，而是双方都需要的结果：芯片厂商需要一个低门槛入口证明芯片能用，开源社区需要一个低摩擦路径覆盖更多硬件。

第二股力量是“统一接口”的理想与“深度调优”的现实之间的矛盾。如果能用一套接口让所有国产芯片暴露算力，上层的引擎开发将变得极为简洁。但现实中，不同芯片在图捕获能力、动态形状支持、低比特原生指令和通信语义上差异显著，用同一套接口

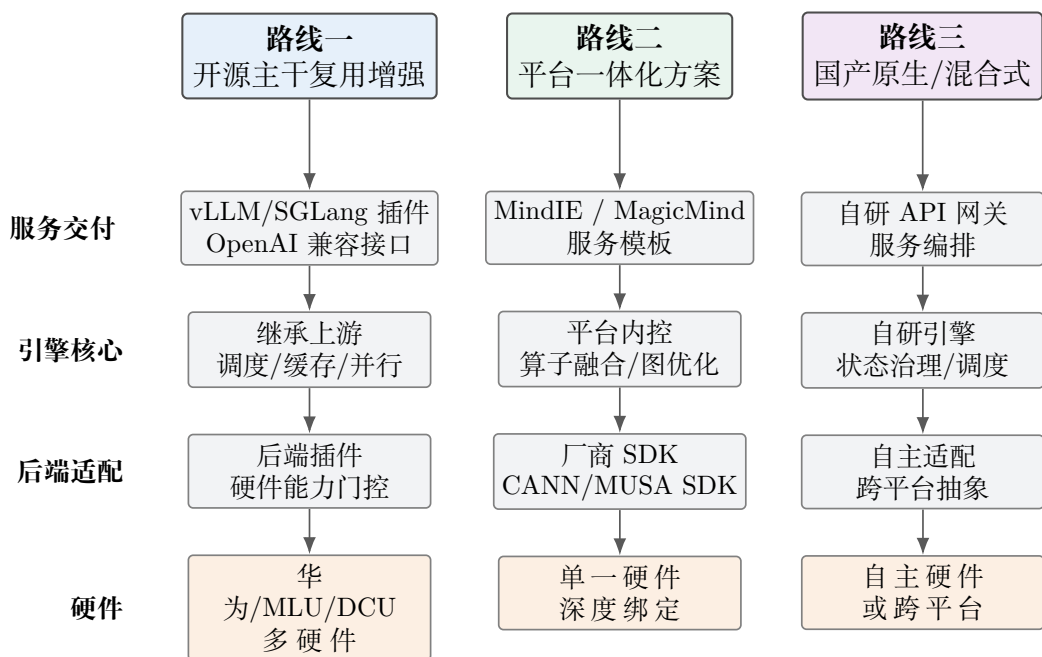
抹平这些差异，几乎总会牺牲某块芯片最擅长的能力。DLInfer、Triton-Ascend 等中间层项目给出了一种折中思路：上层定义与框架交互的稳定接口，下层留给厂商填充原生实现。这套模式能否成熟，将在很大程度上决定国产推理生态是走向收敛，还是继续在多个“插件孤岛”之间分治。

表1 是一张生态快照，汇总了当前公开可查的主要项目与组织。读者不必逐行研读，把它当作印证前文两股力量的参照即可。如果留心观察，会发现一个有趣的规律：生态中没有哪个项目能同时所有维度上都做到最优——每个项目的形态本身，就是它背后的团队在上述两股力量中做出取舍的印记。

表 1: 面向国产算力的大模型推理引擎与适配生态

组织/单位	面向算力	代表项目	公开形态与技术关注点
华为 Ascend	Ascend NPU	MindIE, vLLM-Ascend, SGLang-Kernel-NPU, Triton-Ascend	厂商推理套件 + vLLM 插件 + SGLang NPU kernel + Triton 后端；关注 CANN/torch-npu 适配、MoE kernel、编译后端 [11–14]
上海 AI 实验室	多类国产加速器	DLInfer, LMDeploy	跨厂商解耦层 + TurboMind/PyTorch Engine 双引擎 [15, 16]
百度 / 昆仑芯	Kunlun XPU 等	FastDeploy, vLLM-Kunlun	PD 分离、统一 KV 传输、OpenAI 兼容接口 [17, 18]
寒武纪	MLU	vLLM-MLU, MagicMind	vLLM 插件 + 原生推理加速引擎 [19, 20]
摩尔线程	MUSA GPU	vLLM-MUSA, MT-Transformer	通用兼容插件 + 高性能原生后端双路线 [21, 22]
沐曦	MetaX GPU	vLLM-MetaX	MACA 类 CUDA 后端接入 vLLM[23]
燧原科技	GCU	vLLM-GCU	支持量化、Fused MoE、Speculative Decoding 等 [24]
天数智芯	Iluvatar CoreX	IxRT, IGIE	高性能运行时 + TVM 通用引擎 [25]
海光	Hygon DCU	FastDeploy-DCU	大模型部署示例 [26]
壁仞	BR 系列	BIRENSUPA, EngineX	开发者平台 + vLLM OOT 插件 [27]
瀚博 / Vastai	Vastai GPU	Xinference-VACC	服务化部署平台 [28]
Xinference / GPUStack	多类异构	Xinference, GPUStack	模型服务编排与多硬件管理层 [29, 30]

把这张快照整体看一遍，可以提炼出三个正在发生的变化。首先，适配模式正从源



三条路线的根本差异不在优化手段，而在系统边界和演进节奏的假设不同

图 2：国产推理引擎三类技术路线的生态分化：从硬件到服务交付的四层对比

码级 Fork 和侵入式修改，不可逆地走向插件化与后端解耦。这不是因为“插件化”是某种最佳实践，而是因为上游框架的演进速度已经快到没有任何单一厂商能承受反复合并的代价。其次，厂商原生引擎在算子融合、图优化和低精度路径上仍保有开源插件难以达到的深度优势，但它们的价值定位正在从“全栈自研”收缩为“关键路径的深度调优”。第三，DLInfer、Triton-Ascend 等中间层正在尝试定义一种新的分工方式——框架负责调度与状态管理，中间层定义能力契约，厂商在契约范围内实现各自的最优路径。这三个变化共同指向一个判断：当适配不再停留于单点移植、而是走向插件化与统一接口时，真正区分不同系统的就不再是“支持了哪些硬件”，而是“在统一抽象、状态管理和压缩协同等核心维度上做出了怎样的架构选择”。这正是下一节的分析骨架要回答的问题。

三、分析骨架与核心架构

当一个用户向推理服务发出一条请求时，引擎内部实际上同时在做三件事：执行模型计算（把 token 变成下一个 token）、管理状态（记住已经算过的东西，确定显存怎么分配）、协调服务（决定多个请求谁先谁后，结果如何送回去）。这三件事对应的就是本文“三条主路径”加“一个底座”的分析骨架。在成熟的 CUDA 平台上，这三件事彼此配合得很好，工程师通常不需要同时操心它们之间的关联。但在国产平台上，麻烦往往先出现在模块交界处——图模式回退时调度器知道不知道？多模态的前处理结果该存在哪里？平台插件到底只管设备发现，还是也得负责环境修复？表2 把这个骨架下最核心的技术问题和评价维度做了一个概览。

表 2: 国产推理引擎关键技术点总览

技术主路径	关键技术点	典型评价维度
执行与通信	统一执行 IR、动态图与图模式切换、Prefill/Decode 阶段解耦、推测解码协同、拓扑感知通信运行时、计算-通信重叠	MFU、吞吐、TTFT、TPOT、跨节点扩展效率、回退稳定性
状态治理	Prefix/共享状态索引、PagedAttention 与块化缓存、多级存储驻留、状态生命周期管理、跨节点状态迁移、命中感知调度	KV 命中率、迁移流量、恢复时延、显存/主存/NVMe 占用、长上下文稳定性
压缩与成本协同	混合精度量化、KV 动态量化、稀疏-量化协同、低比特算子路径、质量约束下的单位 token 成本优化	精度保持、吞吐/时延变化、状态容量变化、带宽占用、单位 token 成本
统一底座与交付	接口协议、指标口径、trace/log/metrics、回归验证、第三方测试证据链、环境治理与部署控制面	可复现性、可比较性、升级回归成本、交付成熟度

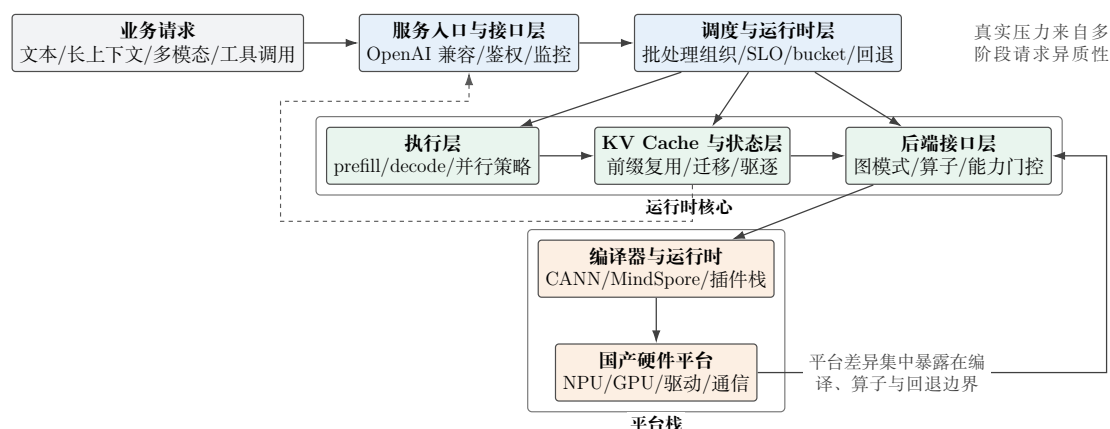
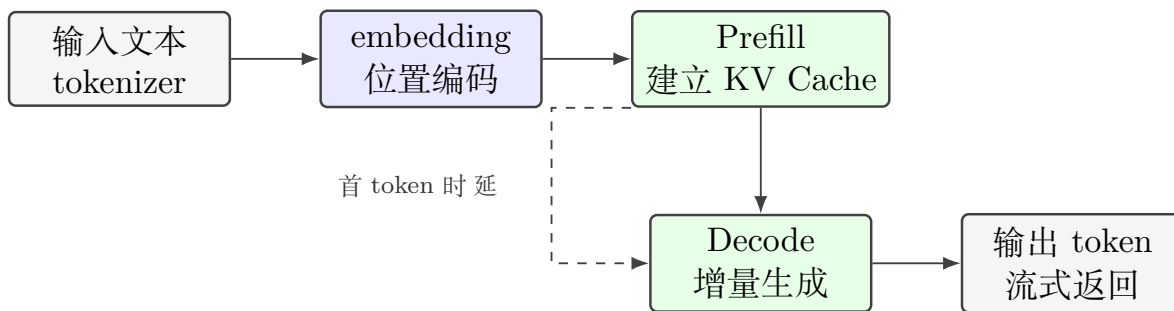


图 3: 国产推理引擎系统设计示意图

3.1 从最小推理闭环到统一分析底座

最简单的推理过程可以想象成两步。第一步是”消化提示词”：系统把用户输入的整段文本并行计算一遍，建立起一个记录了”谁和谁相关”的缓存（KV Cache）——这就是 prefill。第二步是”逐个生成”：每次只算出一个新 token，然后更新缓存——这就是 decode。前者是计算密集型的（像一口气读完一篇文章），后者是带宽密集型的（像一个字一个字地写，每写一个都要回头看一眼前文）。这一差异是所有后续优化的起点：连续批处理、分块预填充、阶段解耦，本质上都在利用 prefill 和 decode 对资源的不同胃口 [5-7]。

理解了 this 最小闭环之后，”底座”的意义就很清楚了：它为多平台、多版本和多阶段优化条件下的比较提供共同语言。如果不同团队对”首 token 时延”的定义都不一样，路线比较就无从谈起；如果缺少持续运行的监控和链路追踪，调度策略到底有没有用就只能凭印象。



稳定生成速度主要受 decode 主
路径、缓存组织和带宽利用影响。

图 4: 从输入到输出的最小推理闭环

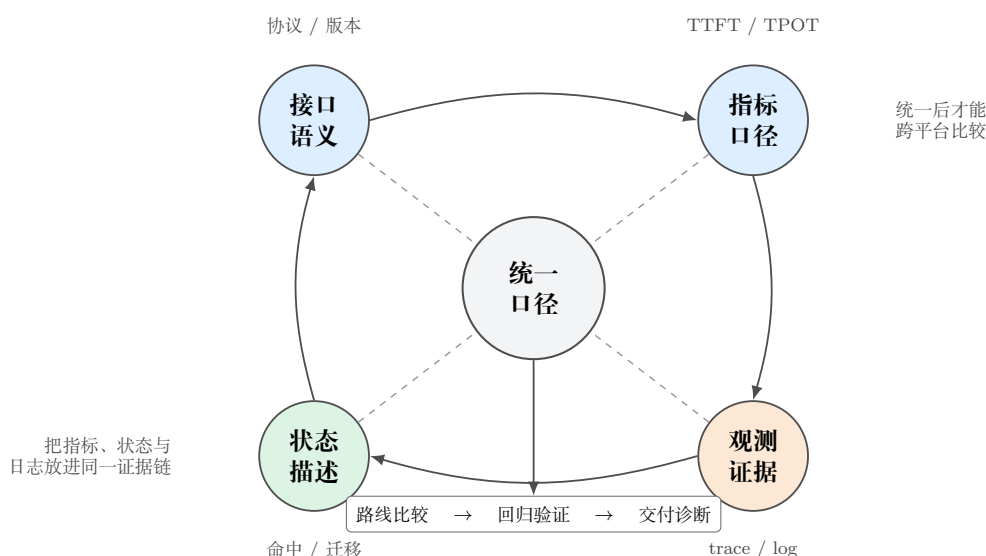


图 5: 可观测性与能力契约层级示意图

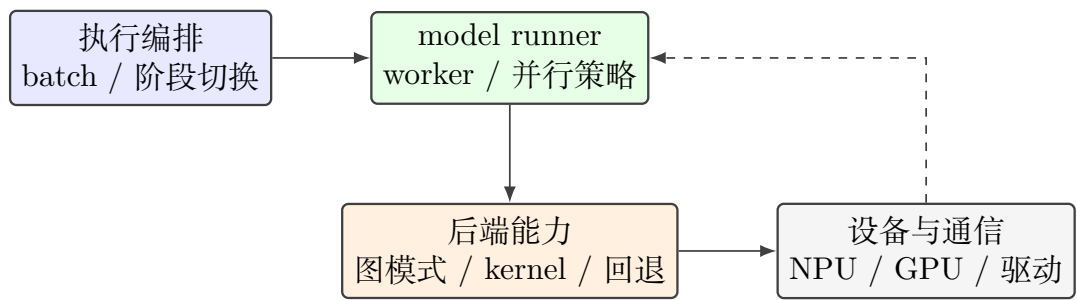
3.2 执行与通信：决定系统能跑多快

执行路径决定的是引擎能把模型算得多快、拓展得多广。要回答的核心问题包括：什么时候让编译器一次性优化整张图（图模式），什么时候回退到逐算子执行（eager 模式）；多张卡之间的通信拓扑怎么组织；以及如何在推测解码、分块预填充和 PD 分离中让计算和通信尽可能重叠起来。

对国产平台而言，执行路径还多了一层操心：维护成本。如果后端为了攒点性能在共享代码路径中塞了大量平台特例判断，一旦上游框架升级了模型执行接口或缓存布局，本地的补丁就会大面积失效。FlashAttention-3 直接利用 H100 的硬件异步执行和 FP8 能力，FlexAttention 则在保留融合算子性能的同时恢复了注意力变体的可编程性 [31, 32]。这类工作揭示的共同规律是：高性能后端的真正难点不在于“有没有 kernel”，而在于“如何把硬件的代际能力组织成可持续复用的接口”。

这就引出了国产平台执行路径的核心设计原则：把平台差异封装成可替换模块，而不是让它们渗透到共享代码中。更重要的是建立一套“后端能力契约”：明确告诉上层，哪些算子可以被图模式安全覆盖，哪些路径必须保留逐算子回退，哪些低精度路径需要调度器显式感知。国产平台的很多问题不是“后端不能用”，而是“某个能力只在特定条件

下才能用”。如果系统没有办法表达这种带条件的能力边界，调度器就只能用越来越多的探测和分支来维持稳定——这正是当前许多国产后端的现状。



平台差异更适合被封装在后端能力边界内，而不是持续回流到共享调度主链。

图 6: 执行后端能力边界与回退路径

3.3 状态管理：记忆怎么存、怎么复用、怎么回收

推理引擎最显著的资源压力不在计算，而在记忆。每一次 decode 都要回头看前文的 KV Cache，而这个缓存随上下文增长会以线性速度占据显存。PagedAttention 的核心思想是把显存像操作系统管理磁盘一样分块管理，避免大块连续分配的浪费 [1]。这一机制同时为前缀复用、分层缓存和跨设备迁移提供了基础。对国产硬件而言，状态系统还面临两个额外困难：不同芯片对内存分配粒度的偏好不同，而且当前缀复用、分块预填充和跨设备迁移同时开启时，缓存元数据的复杂度会急剧上升。

围绕注意力高效实现与缓存压缩已形成清晰技术主线。FlashAttention 系列从算子层逐步降低注意力计算的内存访问代价 [31, 33, 34]；MInference、DuoAttention、vAttention 与 LServe 分别从动态稀疏注意力、检索头/流式头分离、虚拟内存管理和统一稀疏注意力执行改写了长上下文推理的系统组织 [35–38]；H2O、KIVI、SnapKV 与 CacheGen 则从重点点击中、低比特量化、语义压缩和缓存流式传输角度缓解了 KV Cache 的显存与带宽压力 [39–42]。

从更大的视角看，KV Cache 正在从执行层的附属模块演变为独立的基础设施。当长上下文、多轮会话和多 Agent 工作流同时存在时，缓存不仅要回答”数据放在哪”，还要回答”状态怎么计价、怎么回收、怎么复用、怎么迁移”。国产推理引擎在这方面更适合采用”统一抽象、平台特化实现”的方式：上层统一暴露块管理、引用计数、前缀复用和驱逐策略接口，具体的数据排布和搬运留给后端来实现。

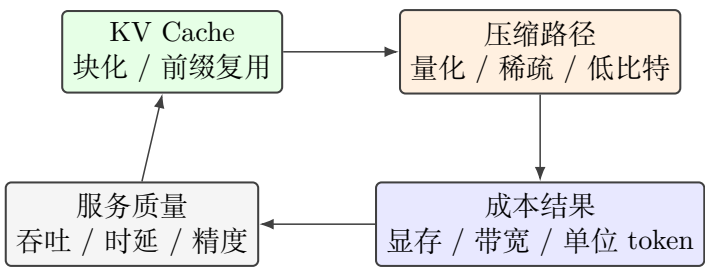


图 7: KV Cache、压缩与成本反馈闭环示意图

3.4 压缩与成本：不只是省显存的事

很多人把量化和压缩看作部署完成后再”省一点显存”的后期技巧。但在国产平台上，压缩应当被视为与执行路径和状态管理并列的核心主路径。原因很简单：混合精度量化、KV 动态压缩和稀疏协同确实能降低显存占用，但它们能不能真正发挥作用，取决于后端有没有稳定的低精度算子路径、状态层能不能处理容量变化、调度器能不能吸收 batch 形态的波动。换句话说，压缩效果从来不由模型精度单独决定，而是由”压缩参数、执行效率、状态占用、服务质量”四个因素共同决定的。

如图7 所示，压缩路径的工程价值需要回到 KV 组织、服务质量和单位 token 成本的联动关系中才能被正确评价。QServe 通过把权重量化、KV 压缩与 serving 系统协同组织，展示了低比特部署必须在 kernel、调度和缓存之间联合设计 [43]；KIVI、SnapKV 则进一步表明，长上下文场景下的 KV 压缩只有和缓存布局与热度判断结合，才可能不显著破坏服务质量 [40, 41]。国产推理引擎若要把”低成本”转化为系统能力，就必须让压缩配置、运行时观测与成本核算使用统一接口。

3.5 服务层收束：调度、接口与交付闭环

服务层需要在吞吐与时延之间平衡。典型机制包括连续批处理、请求优先级控制、SLA 感知调度和流式返回。对国产硬件，还需考虑编译图缓存、静态形状约束与后端初始化成本——调度不仅要优化”请求何时执行”，还要优化”请求如何映射到平台最擅长的执行形态”。XGrammar 说明结构化生成已需要与推理引擎共同设计 [9]；AI Metropolis 进一步表明，多 Agent 负载下依赖感知的乱序执行是提高并行度的关键机制 [44]。

在调度策略方面，DistServe 和 Splitwise 将 Prefill 与 Decode 的资源需求进一步拆解 [7, 8]；Llumnix 通过跨实例动态迁移请求缓解负载失衡，SOLA 则把调度目标显式对齐到 TTFT 和 TPOT 等服务级目标 [45, 46]。TensorRT-LLM 已把多模态支持做成相对完整的运行时能力集合 [47]；OmniServe 尝试把低比特量化与长上下文统一稀疏注意力纳入同一 serving 框架 [48]。这些工作虽不直接针对国产算力，但为理解调度协同、量化部署和工程化取舍提供了对照。

服务做到最后，落脚点是交付闭环。推理引擎面向外部的不只是一个 OpenAI 兼容接口，还包括监控、日志、链路追踪和故障恢复。这一层在国产系统中经常被低估，但在政企部署场景里，系统往往首先因为兼容性、可观测性或升级复杂度不过关而被淘汰。无论是 Transformers 的模型接口、Ray Serve 的服务编排，还是 Triton Inference Server 的部署体系，都在事实上定义了推理引擎需要兼容的工程边界 [49–51]。

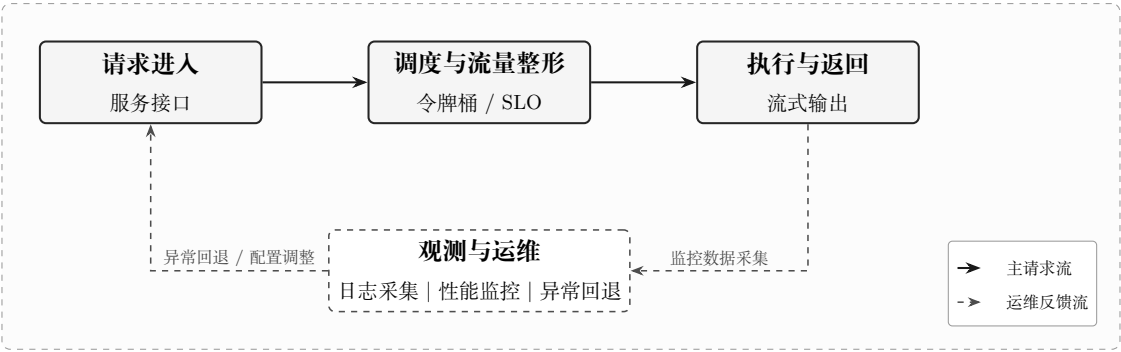


图 8: 调度、服务与运维闭环示意图

把以上三条主路径和底座放在一起看，国产推理引擎的架构难点并不在某个单一模块内部，而是集中在模块与模块的交界处：执行层和编译层的交界决定系统能不能跑，缓存和调度的交界决定长上下文和高并发下表现如何，对外接口和运维的交界决定系统能不能真正上生产。接下来讨论典型技术路线时，重点关注的就是不同系统在这些交界处做出了怎样的取舍。

四、典型技术路线与复杂度分配

回到工程师真实面对的决策场景：时间有限、人力有限、平台约束也有限，团队到底应该把工程力量优先投在哪一层？国产推理引擎当前的三类路线，正好对应了三种不同的答案。

第一种选择是把力量压在执行主路径上：让一个成熟的开源框架尽快在国产后端上跑起来，全面复用社区在调度、缓存和接口上的持续演进。这条路回报最快，但风险也最明确——上游每一次升级都可能在插件边界制造新的摩擦。第二种选择是把力量压在平台栈的整体收拢上：由厂商把推理引擎、编译器、运行时和部署工具打包为一套端到端方案，追求确定性交付。交付质量最高，但前提是团队愿意接受平台定义的边界和节奏。第三种选择则在两者之间找平衡：保持上层接口兼容的同时，通过插件隔离和外部运维工具把国产平台的适配劳动从核心执行链中拆开，既不完全依赖上游，也不被单一平台绑定。

表3 把三类路线的适用前提、典型摩擦和系统收益做了对比。但表格只能呈现静态快照，真正的决策还需要理解每条路线内部的张力是如何演化的——这就是下面几节要展开的内容。

4.1 路线一：把复杂度压在执行主链的开源后端适配

这条路的核心选择是：先让系统跑起来，再说性能。以 vLLM 与 SGLang 为代表的开源推理框架提供了一套久经考验的主干——调度、缓存、结构化输出、多模态前端——国产后端需要做的，表面上只是把执行层从 CUDA 换成自家算子 [2-4, 52]。典型实现方式几乎形成了模板：平台层加设备注册、worker 层注入后端逻辑、热点路径插入定制算子、调度层增加平台约束门控。这条路回报最快，因为上游社区的每一次升级——新模型支持、新调度策略、新压缩路径——后端适配者都可以“搭便车”。但代价也很清楚：一旦抽象边界没守住，每次上游升级都可能把后端 patch 变成需要重新调试的技术债。

在国内部署实践中，这一路线的真实工作量不止于补齐 kernel，还包括模型接入、量化转换、OpenAI 兼容协议、监控埋点和部署脚本等外围工程。后端适配是否成功，往往取决于模型支持矩阵更新速度、异常回退路径是否可观测，以及版本升级时能否继续跟上上游抽象变化。vLLM-Ascend 更能体现对上游执行边界的依赖，LMDeploy 更强调部署工具链与模型接入效率，而 vLLM-HUST 则在上层保持 vLLM 兼容的同时，通过平台插件与外部运行时治理工具把国产平台接入从核心执行链中拆开 [52-54]。这种差异说明，开源主干后端适配内部也在分化为“主链后端实现优先”和“模块边界经营优先”两种形态。

表 3: 国产推理引擎路线选择的前提、摩擦与收益

路线	适用前提	典型摩擦	核心系统收益	适合的团队画像
vLLM-Ascend 类开源后端适配	愿意跟进上游主干演进, 优先保障模型覆盖与生态兼容	版本矩阵对齐、patch 累积、平台特例向共享主链渗透 [52]	快速吸收上游能力, 模型覆盖广	重视生态距离和快速跟进的团队
MindIE 类一体化平台	面向确定性交付, 接受平台约束换取稳定性	生态兼容慢、模型类型受平台节奏限制 [55]	交付确定性高, 模板治理统一	政企落地与稳定交付优先的团队
SGLang 路线	在复杂控制流和结构化输出场景下重视运行时组织优势	需为执行模型与调度抽象投入理解成本 [3, 4]	复杂工作负载调度与编排	复杂工作负载优先的团队
vLLM-HUST 类混合式路线	不愿放弃上游接口兼容, 同时希望逐步沉淀本地治理能力	需持续经营模块边界, 避免继承上游复杂度又堆积本地补丁 [54]	接口兼容、平台特化和运行时治理之间的折中	既要跟进上游又要建设本地化治理的团队

4.2 路线二：把复杂度压在交付链路

这条路的核心选择是：不让用户操心组装，而是由平台统一交付。以 MindIE 为代表，把推理引擎、编译器、运行时与部署工具链整体收拢到同一交付体系内 [11, 55]。其核心理念是：系统从编译到上线应在相同约束空间中统一治理。这条路线在确定性交付和稳定迭代方面具有优势：模型覆盖由平台规划，算子融合深度不受通用接口约束，服务配置可与平台基础设施联动。代价是生态兼容与上游响应速度相对较慢，并且因公开资料有限，外部难以独立验证其能力边界。

4.3 路线三：自研引擎与混合式架构

这条路的核心选择是：自己控制系统边界。Chitu（赤兔）面向多元算力组织弹性推理部署，xLLM 则以国产芯片为中心组织推理内核，两者都没有选择在某个现有框架上“打补丁”，而是从状态机、缓存治理到部署闭环一体自建 [56–58]。好处是清楚的：系统演进不受上游节奏约束，状态治理和资源调度可以为自家硬件量身裁剪。代价也同样清楚：生态吸收速度会慢——社区里每天涌现的新模型、新量化格式和新工具，都需要自行接入而非自动继承。

4.4 多模态与复杂负载对路线分化的加速

如果说前面的讨论还主要围绕文本推理展开，那么多模态和复杂负载的到来则在实质上加速了路线的分化。问题的本质不在于“支持更多数据类型”，而在于当请求从单一文本扩展为包含视觉编码、语音前端、结构化约束验证和工具调用回注等多阶段链路时，三条主路径上的矛盾会同时加剧。

具体而言，多模态推理在三个层次上重写了引擎的假设。首先是 prefill 主路径：多图文档输入导致视觉 token 膨胀和形状剧烈波动，原本为文本 batch 优化的 bucket 划分和编译策略会频繁失效。其次是阶段关系本身：视频理解要求 encode 和 decode 之间插入帧间压缩步骤，文本推理时代“一条请求一个 decode 循环”的模型被打破。第三是会话状态机：语音交互引入流式 chunk 和实时打断，使引擎必须同时管理文本、音频和视觉多种状态对象，而非只维护一个 token 序列。

这些变化对三条路线的影响并不对称。对开源后端适配路线，冲击集中在执行主路径上——每次新模态出现都需要评估后端算子的覆盖度、编译路径的完整性和异常回退

表 4: 国内部署路线案例比较

代表对象	路线类型	工程落点	部署目标
vLLM-Ascend[52]	开源主干后端适配	围绕上游接口扩展后端与定制算子	生态兼容与模型覆盖优先
vLLM-MLU[59]	开源主干后端适配	依托 vLLM plugin system 接入寒武纪 MLU	继承 vLLM 接口与调度语义
SGLang Ascend[60]	主线后端支持	在 SGLang 主线中为 Ascend NPU 提供 attention backend	国产 NPU 被主流框架吸纳
vLLM-HUST[54]	混合式架构	保持上层接口，外挂平台插件与运行时护栏	上游兼容、国产平台接入与运维闭环
MindIE[55]	一体化部署	与本地编译器、运行时深度协同	平台统一交付优先
Chitu[56]	独立自研引擎	面向多元算力组织弹性部署	多硬件兼容与生产可用性
xLLM[57]	自研框架	围绕国产芯片组织推理内核与部署入口	同步建设推理内核与部署闭环
LMDeploy[53]	工具链强化	围绕模型支持、推理工具链持续扩展	高性能部署与工程效率
FastDeploy[61]	工具链强化	以高性能工具包衔接模型导出与服务封装	围绕飞桨生态组织 LLM 部署

的可靠性，而这些正是此类路线最脆弱的环节。对一体化方案，挑战则在生态响应速度——当模型形态以月为单位快速变化时，平台统一规划的模式可能难以同步跟进。对混合式路线，多模态反而为其提供了合法性的论据：既然没有任何单一路线能覆盖所有模态的最优路径，那么”经营模块边界、在关键路径上允许平台特化”就变得比”追求全栈统一”更务实。这一判断也解释了为什么像 vLLM-HUST 这类路线近年受到关注——它不是某种技术偏好的产物，而是多模态时代对系统架构提出的实际要求。

4.5 路线比较与选择框架

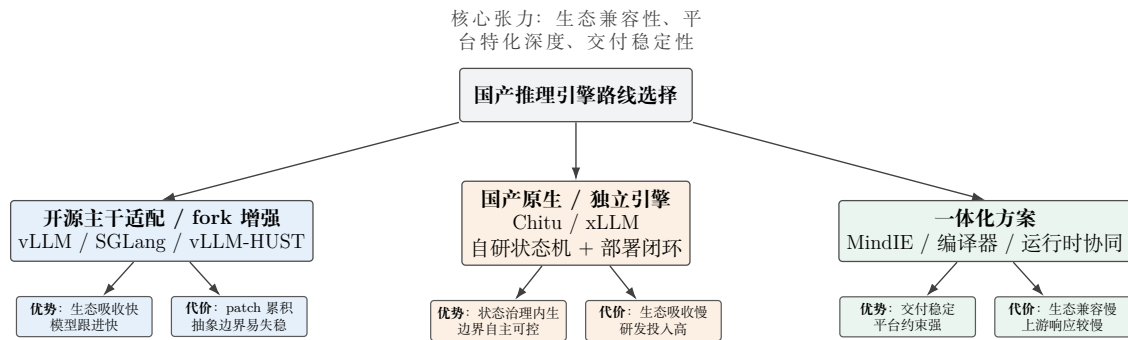


图 9: 国产推理引擎技术路线分化示意图

将上述分析放在一起，可以看到一个更底层的判断逻辑：选路线不是选”哪个更好”，而是选”在我的约束下，哪个维度的优先级应该排在最前面”。团队如果最怕模型覆盖掉队，就应该让执行主路径尽快畅通，其他维度暂且交给社区和厂商分头解决。团队如果最怕交付出事——这在政企场景中极为常见——就应该接受慢一拍的模型跟进节奏，换取编译、运行时和运维的一体化治理。而如果团队的真实处境是”既要跟上游又要攒本地能力”——国内大多数部署团队恐怕都在这个区间——那么混合式路线就成了一种务实的折中，但它能持续成立的前提是：团队有能力在上游接口快速变化和本地治理需求相对稳定之间，维持一条清晰的模块边界。

表 5: 国产推理引擎关键挑战比较

挑战来源	主要冲击层	典型后果
硬件异构与后端碎片化	执行层、算子接口、通信路径	共享路径分叉、正确性边界不一致、维护成本持续上升
复杂任务与多模态负载	调度、缓存、图模式与前处理链路	阶段争用加剧、时延波动放大、端到端不稳定
压缩收益与系统成本错配	量化路径、KV 状态、调度与成本核算	低比特收益不稳定、单位 token 成本难形成闭环
上游快速演进与本地分叉	平台抽象、插件接口与共享模块	patch 累积、合并困难、对新模型和新特性的跟进滞后
评测体系与真实业务脱节	benchmark 设计、运维指标与资源评估	优化方向失真、离线结论与生产表现偏离

除路线本身的选择外，还有一个容易被忽略的维度正在成为新的分化点：多租户与复杂状态空间下的运行时组织能力。当同一个引擎实例需要同时服务数十个 LoRA 适配器、管理多个模型版本的 KV 共享内存池、并在不同优先级的多 Agent 工作流之间动态分配计算资源时，“调度的粒度”就不再是 batch 尺寸调优，而是运行时是否具备对状态对象进行统一建模和动态编排的能力。这组问题超出了单一框架或平台的范围，直接指向下一节要讨论的核心——当这些架构选择在国产平台的实际部署中相互碰撞时，哪些结构性挑战会首先暴露出来。

五、关键挑战：主路径如何相互放大

一个常见的场景可以说明问题的复杂性：团队好不容易让某款开源模型在国产后端上跑通了推理，吞吐指标也看得过去。但接下来做量化时发现，低比特算子在该后端上行为不一致，于是绕了一圈换方案；换完之后 KV Cache 管理逻辑也要跟着改，改了又触发了调度策略里一个隐式假设，导致尾时延飙升；最后想对比一下优化前后的效果，却发现团队用的 benchmark 根本没覆盖这种混合精度加长上下文的场景。这不是偶发故障，而是国产推理引擎面临的结构性困境：挑战之间会相互放大，一个路径上的适配缺口会沿着系统骨架向其他路径扩散。表5 把这种放大关系压缩为总览。

表6 进一步说明：不同路线并非面对同一组问题再做不同优化，而是会把”版本矩阵、网络控制面、缓存/调度、运维回退”这些挑战优先暴露在不同层次上。换句话说，选路线的那一刻，就已经在决定团队首先要把系统能力压到哪一层。社区部署记录也反复印证了这一点。CANN/torch_npu 版本矩阵对齐之难（vLLM-Ascend 0.7.x 到 0.8.x 升级期间多次出现 HCCL 通信与算子兼容性问题）、PD 分离与双机组网的稳定性焦虑、MindIE 服务化配置的可控性争议、LMDeploy 与 vLLM 在国产 GPU 环境中的适配取舍——这些反复出现的摩擦点，追根溯源都能回收到表5 列出的结构性挑战上。真实摩擦往往先出现在环境治理和平台工具链层面，而不是单点跑分上。下面沿前文三条主路径，逐一讨论这些挑战如何沿系统骨架扩散。

表 6: 不同国产推理路线与关键挑战的映射关系

代表路线	版本矩阵/环境门槛	网络与分布式风险	缓存/调度压力	运维与回退焦点	更需要优先建设的系统能力
vLLM-Ascend 类开源后端适配	高。上游主干、插件分支、CANN 与 torch_npu 对齐要求强 [52]	中到高。PD 分离与多机连通性常在扩容时暴露	中。继承上游调度能力，但平台约束可能导致命中率下降	版本锁定与环境预检	持续回归基线与插件边界稳定性
MindIE 类一体化平台	中。平台内控但对外兼容性需额外验证 [55]	低到中。平台内一体治理	中。内部调度但缺乏外部可观测证据	模板敏感性与配置可控性	服务模板的可诊断性与生态兼容接口
Chitu / xLLM 类自研路线	中。自主控制但生态距离远 [56, 57]	取决于自研通信层成熟度	高。状态治理需自建	全栈自主但生态覆盖有限	模型覆盖速度与社区生态对接
vLLM-HUST 类混合式路线	中到高。继承上游版本压力，同时维护本地插件层 [54]	中。继承上游分布式能力，补充平台护栏	中。上游调度加本地门控	模块边界维护与 merge-safe 演进	持续经营模块边界防止双重分叉

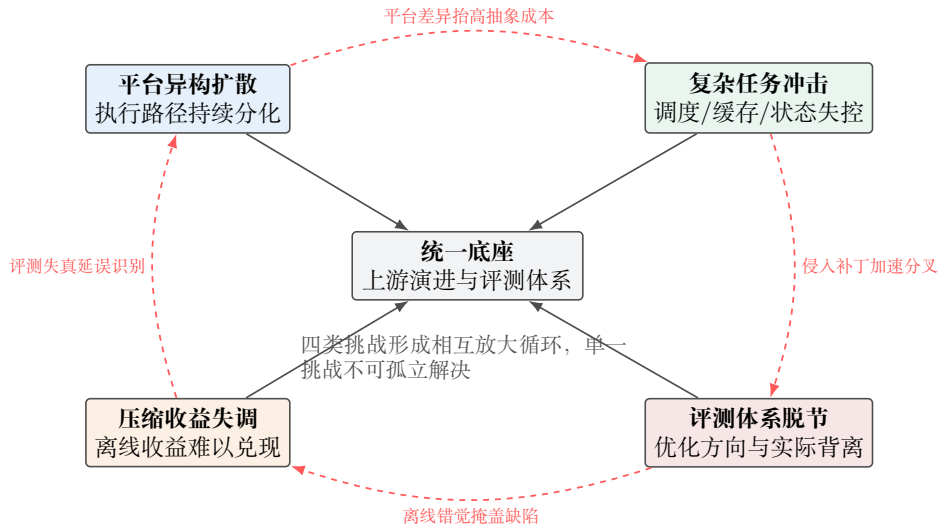


图 10: 关键挑战的相互放大关系：四类结构性问题通过统一底座形成闭环反馈

5.1 执行与通信主路径：平台异构的持续扩散

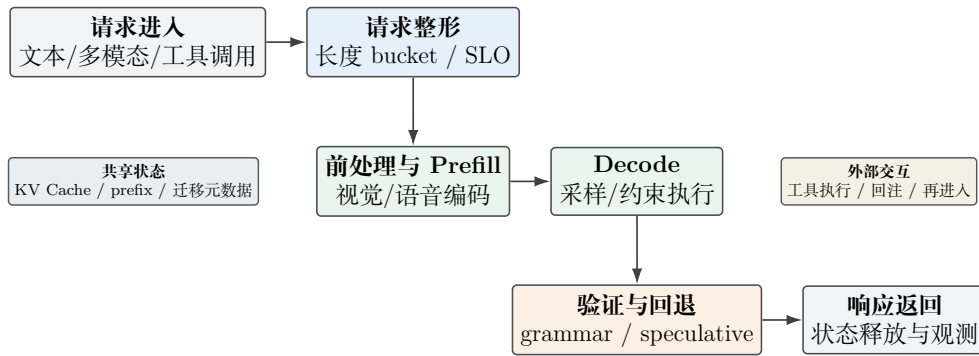
第 3 节已将后端能力契约列为执行路径的核心问题，这里要追问的是：为什么平台异构在国产推理引擎中不是“适配一次就完”的事？

答案在于硬件代际能力差异大且缺少统一适配层。vLLM-Ascend、vLLM-MLU 与 SGLang Ascend 的公开材料都体现出对上游主线版本、CANN SDK 和 torch_npu 三方联合对齐的强依赖 [52, 59, 60]。这意味着每当上游发版、SDK 升级或 PyTorch 变更 API，三个环节都必须同步验证——任何一个掉链子，整体就可能退回到不可用状态。更麻烦的是，不同硬件上的 feature parity 不能默认成立：graph mode、attention backend、量化路径、PD 分离和 speculative decoding 等能力，即使在同一工具链中也可能存在支持粒度差异 [62, 63]。这就解释了为什么社区经常看到“某个功能在 A 卡上能跑，在 B 卡上就不行”的报告——不是 bug，而是能力边界还没收敛成稳定契约。

5.2 状态治理主路径：复杂任务场景的能力补齐

如果说执行路径的问题是“跑不跑得动”，那么状态治理路径的问题就是“记不记得住、调不调得好”。当前真实负载已远不止单轮文本生成——长上下文、多模态、结构化输出、工具调用和多 Agent 协同 [9, 10, 44]，每一种都在改变“状态”这个词的含义。

文本 serving 时代，系统管理的状态对象相对简单：请求队列、KV Cache 和 batch 形状。进入多模态和工具调用场景后，运行时还需管理视觉前缀、语法约束状态、工具回注上下文、外部执行等待与流式中断恢复——它们既非纯粹的模型内部变量，也无法完全交给服务层处理，而是直接影响调度顺序、缓存复用和回退路径选择的运行时实体。结构化输出要求把约束执行压进采样环路，工具调用要求支持生成与外部执行之间的往返切换，多模态请求则要求消化更长的前处理链和更剧烈的 shape 波动。对国产平台，难点不在接口名义上“可用”，而在这些能力会同时冲击调度、缓存和图模式三个层次。



复杂请求的关键难点不在单步执行，而在跨阶段状态传递、资源复用与回退一致性。

图 11: 复杂请求生命周期示意图

推测解码进一步抬高了要求。SpecInfer、Medusa、EAGLE 与 Lookahead Decoding 分别代表辅助模型、多头预测、特征不确定性建模与并行前瞻路线 [64–68]。但收益高度条件化：在 MI300X 上的实验表明，speculative decoding 主要受益于小 batch 的带宽受限 decode，batch 增大后收益明显收缩 [69]。对国产平台，一个“只是多一次验证”的机制，迁移过来后可能同时意味着图缓存失效和异常回退链变长——系统复杂度的增长远超表面看到的那一步。

5.3 压缩与成本主路径：收益不稳定与闭环缺失

压缩路径的麻烦在于，它并不是一个可以独立解决的问题——它和执行路径、状态治理直接纠缠在一起。执行路径上的平台异构会限制低比特算子的可用边界，状态治理层的缓存组织则决定压缩策略能否在运行时稳定兑现。三者缺一，压缩收益就只能停留在离线实验阶段。

“低成本”听上去像是部署最后一步才需要关心的经济性指标，其实不然——它与执行效率和状态治理同级。问题在于压缩策略的收益往往非线性兑现：同一组量化参数在不同后端、不同上下文长度和不同负载强度下，可能分别表现为吞吐提升、尾时延恶化或精度波动。QServe 说明只有把量化、kernel 组织和 serving runtime 一起设计，低比特收益才可能在真实服务路径中落地 [43]；KIVI 与 SnapKV 表明长上下文场景下的状态压

缩如果不和恢复路径与质量门限一起考虑，容量节省就会转化为恢复时延或命中率损失 [40, 41]。在国产平台上这种耦合还会被放大——许多后端对低比特算子、动态 shape 和异步搬运的支持边界互不一致，让原本在 NVIDIA GPU 上验证过的压缩策略未必能直接迁移。

5.4 统一底座：上游演进与评测体系

如果说前面三条路径上的挑战各有侧重，那么”统一底座”层面的问题则是它们共同的放大器。底座不只是技术架构中的抽象层，更指系统在持续演进中能否维持工程秩序——这取决于两件事：与上游的关系怎么处理，以及用什么来度量系统的真实状态。

先说上游演进。高性能推理框架仍在快速变化，国产后端如果通过侵入式修改共享路径获得短期收益，后续将面临高昂合并成本。本地分叉之所以危险，还有一层更隐蔽的原因——它会逐步改变团队的优化激励。若长期依赖”复制上游模块 + 本地修补”的模式，团队最容易积累的是”快速打补丁”的经验，而非”设计稳定扩展面”的能力。反过来，若平台特化更多收敛在插件、注册表和受控配置门面中，团队就更有机会把每次适配劳动沉淀为可复用的工程机制。所以 merge-safe 不只是”对上游友好”的姿态，而是国产推理引擎能否持续演进的核心条件。

再说评测。当前不少优化仍围绕静态 benchmark 展开，但真实业务包含请求长度分布变化大、多租户并发、冷热模型混部和 SLA 约束严格等特征。如果评测只覆盖理想化场景，那么团队优化的方向和生产环境的真实需求就会持续脱节。未来评测需要从”单点性能测试”扩展为”端到端工作负载评测”，将 TTFT、稳定吞吐、缓存命中率、异常恢复时间和资源成本纳入统一框架。OpenCompass 与 FlagPerf 分别代表了模型能力评测和系统性能评测的生态努力 [70, 71]。真正有价值的评测应当把这两类视角连起来——既看模型效果，也看支撑这些效果付出了多少系统成本。评测体系需要分层：底层算子用 microbenchmark 验证，运行时调度用受控 serving workload 验证（重点是 TTFT、TPOT 与尾时延），平台演进能力和运维代价则更适合通过持续集成基线、版本升级回归和故障演练来评估。

六、未来研究展望

前一节把路线比较收束为若干系统挑战，这些挑战如果长期得不到结构性回应，就会持续限制国产推理引擎的演进上限。那么未来真正值得投入的方向在哪里？我们的判断可以压缩为三条主线：面向异构平台的能力契约与共享抽象、面向真实交付的控制面与证据链治理，以及面向复杂请求图的状态中心化运行时。图12 概括了三条主线在国产推理生态中的收敛关系。

这三条主线并不是彼此独立的新功能列表，而是前文”一个底座、三条主路径”在未来更可能沉淀出的高层研究议题。它们之间存在递进关系：能力契约为系统演进提供稳定边界，控制面治理让这些边界变得可验证、可交付，而请求图重构则要求前两者同时升级以适应更复杂的运行时状态。



图 12: 国产推理引擎未来演进方向关系图

6.1 面向异构平台的能力契约与共享抽象

这条主线的核心问题是：平台差异能否从“各家项目内部的经验”上升为“生态可以共享的能力契约”。

目前的现状是，后端能力边界通常以隐式特判存在——代码里散落着各种 if-else，却缺少一套统一的描述语言来表达“这个后端能做什么、不能做什么”。要改变这一局面，首先需要回答的是能力应如何形式化表达：既要覆盖 graph/eager 约束、通信语义和低比特路径的真实差异，又不能把某个短期平台特例固化为长期接口。其次是回退语义的定义问题——当前系统对回退的处理通常是“降级到 eager”，但 graph mode 切换、缓存状态和分布式执行在回退前后的一致性，并没有系统性保证。第三个问题则关乎抽象的厚薄：若共享抽象过薄，平台差异反复溢出到应用层；若过厚，又把当前的短期特例固化为未来的包袱。Llumnix、SOLA、LServe 等工作之所以值得关注，正因为它们显示运行时组织方式正在脱离“单一模型 serving 外壳”，转向资源与状态中枢的形态 [38, 45, 46]。

只有当平台差异能够被收敛为共享能力矩阵——而非反复改写系统主链——新模型、新后端和新任务形态的进入才会从高风险迁移变成稳定的生态扩展。然而，能力契约本身并不能自动保证这些边界在生产环境中被持续遵守——这需要控制面来承担验证、回归和交付的闭环责任。

6.2 控制面、评测与交付的一体化治理

这条主线的核心问题是：控制面能否从运维脚本的集合演化为连接评测、成本与交付的一体化治理层。

从当前实践看，几个缺口最为明显。第一是近真实工作负载的标准化表达问题：目前缺少能同时覆盖请求分布、上下文长度变化、多模态混合和工具调用比例的统一 workload 描述格式。BurstGPT 等工作已开始提供真实负载数据集 [72]，但从数据集到可复现评测协议之间仍有显著距离。第二是版本基线与故障注入的统一问题——升级回归和灰度成本目前无法被量化治理，大多停留在人工经验阶段，让每次升级都像一场“盲飞”。第三是模型效果、系统成本与交付代价之间的证据链尚未连通，导致“性能结论”和“生产结论”长期脱节：论文里跑分好看的方案，到了生产环境可能因为运维成本过高而不可持续。

概括来说，执行面负责把 token 一个个生成出来，控制面则负责让 token 生成处于可比较、可复现、可灰度和可回滚的条件之下。只有把控制面视为与执行面并列的一等

对象，前文讨论的成本、评测与交付问题才可能从局部经验转化为稳定的生态能力。而控制面治理的成熟度，反过来也取决于运行时本身能否提供更清晰的状态对象——这正是第三条主线关注的问题。

6.3 面向多阶段请求图的运行时重构

这条主线的核心问题是：当文本 decode、视觉 encode、语音前端、结构化约束验证、工具调用回注和多 Agent 协同被纳入同一服务链路后，推理引擎如何从维护单一 token 生成循环转向管理多阶段请求图。

这个转变并非遥远——DeepSeek-V2/V3 的 MLA 与 MoE 组织连同 FlashMLA、Mooncake 等配套工作，已经把通信组织、缓存布局和注意力实现重新推回运行时中心位置 [73–76]。这要求状态对象不再只按模型名单组织，而应围绕阶段关系来组织：哪些阶段可以并行、哪些阶段之间存在数据依赖、哪些中间产物值得缓存和共享。同样的逻辑也体现在模型侧——Qwen2-VL、InternVL2、GLM-4-Voice 与 MiniCPM-o 等多模态模型已经说明，视觉、视频和语音链路使文本时代的单一路径假设越来越难以维持 [77–80]。当新任务形态进入时，系统不应需要重写主链——理想状态是通过声明新的阶段关系和状态类型，让运行时自动完成调度和资源分配的适配。

这个转变未必立刻带来最亮眼的跑分，但更可能决定国产推理系统能否在未来几年内持续吸收新结构、新任务和新平台。系统边界会被模型公司、应用形态和硬件路线共同牵引，任何一方的变化都会迫使另外两方重写分工。

回过头来看，这三条主线共同指向同一个判断：国产推理引擎的下一阶段竞争，将从“谁更快适配某个硬件后端”转向“谁能在异构平台、复杂负载和快速演进之间建立更稳定的工程秩序”。能力契约、控制面治理和请求图重构不是三个孤立议题，而是同一问题的三个切面——系统能否把持续增加的复杂度组织进可扩展的边界，而不是让它们在外围不断堆积。

七、结束语

本文沿着“生态格局—分析骨架—路线比较—关键挑战—收敛方向”这条线索，对国产算力平台上的大模型推理引擎做了一次结构化梳理。如果要把全文的核心判断浓缩为一句话，那就是：国产推理引擎的竞争点不在局部算子快不快，而在于系统能否在同一架构中同时承受后端异构、复杂任务负载和上游快速演进三类压力，并在国产芯片与本土部署环境中形成可持续的工程闭环。

这一判断落在四个层面上。第一，平台异构的真正终点不是“兼容了多少后端”，而是硬件差异能否被收敛为稳定、可验证、可回退的能力契约——若能力边界长期以隐式特判存在，新后端的接入就会反复冲击共享主链。第二，长上下文、多模态、结构化输出与 Agent 负载正在把推理引擎从 token 生成器推向状态中心化运行时，状态治理能力将比局部算子优化更能决定系统上限。第三，量化与 KV 压缩只有进入统一的观测、调度和成本闭环，才可能从离线实验转化为可持续的单位 token 成本优势。第四，路线选择本质上是复杂度分配——复杂度由 backend、runtime、state machine 还是控制面承担，将直接决定系统的演进速度与长期维护成本。

面向未来，下一阶段的竞争不在单点峰值还能提高多少，而在于谁能更快、更稳、更

低成本地吸收模型变化并完成生产交付。对产业侧而言，建设重点应从围绕单模型做专项优化转向围绕持续演进负载建设系统能力；对研究侧而言，则需要把平台迁移成本、长周期维护成本和近真实服务指标放到与性能同等重要的位置。沿这些方向持续推进，国产推理引擎的演进才更可能从阶段性适配走向可持续积累。

金海 CCF 会士、副理事长。华中科技大学计算机学院教授。主要研究方向为计算机体系结构、计算系统虚拟化、集群计算和云计算、并行 I/O。hjin@hust.edu.cn

参考文献

- [1] Woosuk Kwon et al. Efficient memory management for large language model serving with pagedattention. *Proceedings of the ACM Symposium on Operating Systems Principles*, 2023.
- [2] vLLM Team. vllm project. <https://github.com/vllm-project/vllm>, 2024.
- [3] SGLang Team. Sglang project. <https://github.com/sgl-project/sglang>, 2024.
- [4] Lianmin Zheng et al. Sglang: Efficient execution of structured language model programs, 2023. arXiv:2312.07104.
- [5] Gyeong-In Yu et al. Orca: A distributed serving system for transformer-based generative models, 2023. arXiv:2309.06180.
- [6] Amey Agrawal et al. Sarathi-serve: Efficient LLM inference by piggybacking decodes with chunked prefills, 2023. arXiv:2308.16369.
- [7] Yongzhe Zhong et al. Distserve: Disaggregating prefill and decoding for goodput-optimized LLM serving, 2024. arXiv:2401.09670.
- [8] Harsh Patel et al. Splitwise: Efficient generative LLM inference using phase splitting, 2024. arXiv preprint.
- [9] Yixin Dong, Charlie F. Ruan, Yaxing Cai, Ruihang Lai, Ziyi Xu, Yilong Zhao, and Tianqi Chen. Xgrammar: Flexible and efficient structured generation engine for large language models, 2024. MLSys 2025, arXiv:2411.15100.
- [10] Zedong Liu, Shenggan Cheng, Guangming Tan, Yang You, and Dingwen Tao. Elasticmm: Efficient multimodal llms serving with elastic multimodal parallelism, 2025. NeurIPS 2025 Oral.
- [11] Huawei Ascend. Mindie documentation. <https://www.hiascend.com/document/detail/zh/mindie/100/index/index.html>, 2026. Accessed: 2026-05-07.
- [12] vLLM Project. vLLM Ascend: Community-maintained hardware plugin for ascend npu. <https://github.com/vllm-project/vllm-ascend>, 2026. Accessed: 2026-05-07.

- [13] SGLang Project. SGLang Kernel NPU: Kernel library for ascend npu. <https://github.com/sgl-project/sgl-kernel-npu>, 2026. Accessed: 2026-05-07.
- [14] Ascend. Triton-Ascend: Triton backend for ascend npu. <https://github.com/Ascend/triton-ascend>, 2026. Accessed: 2026-05-07.
- [15] DeepLink Org. DLinfer: Connecting domestic hardware to llm inference frameworks. <https://github.com/DeepLink-org/dlinfer>, 2026. Accessed: 2026-05-07.
- [16] InternLM / OpenMMLab. LMDeploy: A toolkit for compressing, deploying, and serving llms. <https://github.com/InternLM/lmdeploy>, 2026. Accessed: 2026-05-07.
- [17] PaddlePaddle. FastDeploy: Production-ready deployment toolkit for llms and vlms. <https://github.com/PaddlePaddle/FastDeploy>, 2026. Accessed: 2026-05-07.
- [18] Baidu. vLLM-Kunlun: vllm hardware plugin for kunlun xpu. <https://github.com/baidu/vLLM-Kunlun>, 2026. Accessed: 2026-05-07.
- [19] Cambricon. vLLM-MLU: vllm plugin for cambricon mlu. <https://github.com/Cambricon/vllm-mlu>, 2026. Accessed: 2026-05-07.
- [20] Cambricon. MagicMind: Inference acceleration engine for cambricon mlu. https://github.com/Cambricon/magicmind_cloud, 2026. Accessed: 2026-05-07.
- [21] Moore Threads. vLLM-MUSA: vllm hardware plugin for moore threads musa gpus. <https://github.com/MooreThreads/vllm-musa>, 2026. Accessed: 2026-05-07.
- [22] Moore Threads. vLLM-MTT: vllm backend based on mt-transformer. <https://docs.mthreads.com/mtt/mtt-doc-online/>, 2026. Accessed: 2026-05-07.
- [23] MetaX-MACA. vLLM-MetaX: vllm hardware plugin for metax gpus. <https://github.com/MetaX-MACA/vLLM-metax>, 2026. Accessed: 2026-05-07.
- [24] Enflame Technology. vLLM-GCU: vllm adaptation for enflame gcu. <https://github.com/EnflameTechnology/vllm-gcu>, 2026. Accessed: 2026-05-07.
- [25] Iluvatar / DeepSpark. DeepSparkInference: Inference examples on iluvatar hardware. <https://openi.pcl.ac.cn/iluvatar/ZhiKai100>, 2026. Accessed: 2026-05-07.
- [26] PaddlePaddle. FastDeploy deployment on hygon dcu. https://paddlepaddle.github.io/FastDeploy/zh/get_started/installation/hygon_dcu/, 2026. Accessed: 2026-05-07.
- [27] Biren Technology. Biren developer community. <https://developer.birentech.com/>, 2026. Accessed: 2026-05-07.

- [28] Vastai Technologies. Vastai developer center. <https://vastai-tech.github.io/>, 2026. Accessed: 2026-05-07.
- [29] Xorbits. Xinference: Heterogeneous inference platform documentation. <https://inference.readthedocs.io/zh-cn/latest/>, 2026. Accessed: 2026-05-07.
- [30] GPUStack. GPUStack: Open-source gpu cluster manager for ai model deployment. <https://github.com/gpustack/gpustack>, 2026. Accessed: 2026-05-07.
- [31] Jay Shah, Ganesh Bikshandi, Ying Zhang, Vijay Thakkar, Pradeep Ramani, and Tri Dao. Flashattention-3: Fast and accurate attention with asynchrony and low-precision, 2024. NeurIPS 2024, arXiv:2407.08608.
- [32] Juechu Dong, Boyuan Feng, Driss Guessous, Yanbo Liang, and Horace He. Flexattention: A programming model for generating fused attention variants. *Proceedings of Machine Learning and Systems*, 2025.
- [33] Tri Dao et al. Flashattention: Fast and memory-efficient exact attention with IO-awareness. *Advances in Neural Information Processing Systems*, 2022.
- [34] Tri Dao et al. Flashattention-2: Faster attention with better parallelism and work partitioning. *International Conference on Learning Representations*, 2024.
- [35] Hanqing Jiang et al. Minference 1.0: Accelerating pre-filling for long-context LLMs via dynamic sparse attention, 2024. NeurIPS 2024 Spotlight, arXiv:2407.02490.
- [36] Guangxuan Xiao et al. Duoattention: Efficient long-context LLM inference with retrieval and streaming heads, 2024. arXiv:2410.10819.
- [37] Woosuk Kwon et al. vattention: Dynamic memory management for serving LLMs without pagedattention, 2024. arXiv:2405.04437.
- [38] Shang Yang, Junxian Guo, Haotian Tang, Qinghao Hu, Guangxuan Xiao, Jiaming Tang, Yujun Lin, Zhijian Liu, Yao Lu, and Song Han. Lserve: Efficient long-sequence llm serving with unified sparse attention, 2025. MLSys 2025, arXiv:2502.14866.
- [39] Zhenyu Zhang et al. H2o: Heavy-hitter oracle for efficient generative inference of large language models, 2023. arXiv:2306.14048.
- [40] Zirui Liu et al. Kivi: A tuning-free asymmetric 2bit quantization for KV cache, 2024. arXiv:2402.02750.
- [41] Yuhui Li et al. Snapkv: Llm knows what you are looking for before generation, 2024. arXiv:2404.14469.
- [42] Yibo Zhang et al. Cachegen: KV cache compression and streaming for fast large language model serving, 2024. SIGCOMM 2024, arXiv:2310.07240.

- [43] Lei Wang et al. Qserve: W4a8kv4 quantization and system co-design for efficient LLM serving, 2024. arXiv:2405.04532.
- [44] Zhiqiang Xie, Hao Kang, Ying Sheng, Tushar Krishna, Kayvon Fatahalian, and Christos Kozyrakis. Ai metropolis: Scaling large language model-based multi-agent simulation with out-of-order execution, 2025. MLSys 2025.
- [45] Biao Sun, Ziming Huang, Hanyu Zhao, Wencong Xiao, Xinyi Zhang, Yong Li, and Wei Lin. Llumnix: Dynamic scheduling for large language model serving, 2024. OSDI 2024, arXiv:2406.03243.
- [46] Ke Hong, Xiuhong Li, Lufang Chen, Qiuli Mao, Guohao Dai, Xuefei Ning, Shengen Yan, Yun Liang, and Yu Wang. Sola: Optimizing slo attainment for large language model serving with state-aware scheduling, 2025. MLSys 2025.
- [47] NVIDIA. Multimodal support in tensorrt llm. [NVIDIA TensorRT-LLM documentation](#), 2026.
- [48] MIT HAN Lab. Omniserve: Unified and efficient inference engine for large-scale llm serving. [GitHub repository](#), 2025.
- [49] Hugging Face. Transformers. [GitHub repository](#), 2025.
- [50] Ray Team. Ray serve documentation. [Ray Serve documentation](#), 2025.
- [51] NVIDIA. Nvidia triton inference server. [GitHub repository](#), 2025.
- [52] vLLM Ascend Team. vllm ascend project. <https://github.com/vllm-project/vllm-ascend>, 2025.
- [53] OpenMMLab. LMDeploy. <https://github.com/InternLM/lmdeploy>, 2025.
- [54] vLLM-HUST Team. vLLM-HUST. [GitHub repository](#), 2026.
- [55] Huawei Ascend. MindIE: Model inference engine. <https://www.hiascend.com/>, 2025.
- [56] thu-pacman. Chitu 「赤兔」. <https://github.com/thu-pacman/chitu>, 2025. GitHub repository, accessed 2026-05-07.
- [57] xLLM Team. xLLM. <https://xllm.readthedocs.io/zh-cn/latest/>, 2026. 官方文档, accessed 2026-05-07.
- [58] xLLM Team. xLLM technical report, 2025. arXiv:2510.14686.
- [59] Cambricon. vLLM-MLU. <https://github.com/Cambricon/vllm-mlu>, 2026. Apache-2.0 GitHub repository, accessed 2026-05-08.

- [60] SGLang Team. Ascend NPUs in SGLang. https://docs.sglang.ai/platforms/ascend_npu.html, 2026. Official platform documentation, accessed 2026-05-08.
- [61] PaddlePaddle. FastDeploy. <https://github.com/PaddlePaddle/FastDeploy>, 2026. High-performance inference and deployment toolkit for LLMs and VLMs, accessed 2026-05-07.
- [62] FastDeploy Team. FastDeploy documentation. <https://paddlepaddle.github.io/FastDeploy/>, 2026. Official documentation, accessed 2026-05-08.
- [63] LMDeploy Authors. LMDeploy supported models. https://lmdeploy.readthedocs.io/en/stable/supported_models/supported_models.html, 2026. Official supported-model matrix, accessed 2026-05-08.
- [64] Xupeng Miao et al. Specinfer: Accelerating generative large language model serving with speculative inference and token tree verification, 2023. arXiv:2305.09781.
- [65] Yaniv Leviathan et al. Fast inference from transformers via speculative decoding. *International Conference on Machine Learning*, 2023.
- [66] Tianle Cai et al. Medusa: Simple llm inference acceleration framework with multiple decoding heads, 2024. arXiv:2401.10774.
- [67] Yuhong Li et al. Eagle: Speculative sampling requires rethinking feature uncertainty, 2024. arXiv:2401.15077.
- [68] Yichao Fu et al. Break the sequential dependency of LLM inference using lookahead decoding, 2024. arXiv:2402.02057.
- [69] Sonali Singh, Karthik Sangaiah, Shenrun Zhang, Ryan Swann, and Ganesh Dasika. Accelerating llm inference: Up to 3x speedup on MI300X with speculative decoding. [AMD ROCm blog post](#), 2025.
- [70] OpenCompass Team. Opencompass. [GitHub repository](#), 2025.
- [71] FlagOpen. Flagperf. [GitHub repository](#), 2025.
- [72] Yuxin Wang et al. Burstgpt: A real-world workload dataset to optimize LLM serving systems, 2025. DOI:10.1145/3711896.3737413.
- [73] DeepSeek-AI et al. Deepseek-v2: A strong, economical, and efficient mixture-of-experts language model, 2024. arXiv:2405.04434.
- [74] DeepSeek-AI. Deepseek-v3 technical report, 2024. arXiv technical report.
- [75] DeepSeek-AI. Flashmla. [GitHub repository](#), 2025.

- [76] Xupeng Qin et al. Mooncake: A KVcache-centric disaggregated architecture for LLM serving, 2024. arXiv:2407.00079.
- [77] Qwen Team. Qwen2-vl technical report, 2024. arXiv technical report.
- [78] Zhe Chen et al. Internvl2: Better multimodal foundation models with stronger vision-language alignment, 2024. arXiv preprint.
- [79] Aohan Zeng et al. Glm-4-voice: Towards intelligent and human-like end-to-end spoken chatbot, 2024. arXiv:2412.02612.
- [80] Junbo Cui et al. Minicpm-o 4.5: Towards real-time full-duplex omni-modal interaction, 2026. arXiv:2604.27393.

A Survey of Large-Model Inference Engines on Domestic Computing Platforms

Zhang Shuhao, Liu Shifeng, Du Yufeng, Ma Junhao, Qiu Ruijie, Liao Xiaofei, Jin Hai

Abstract This survey examines large-model inference engines on domestic computing platforms. Using cross-checkable public evidence, it explains why the ecosystem has split into three routes—upstream-core reuse and enhancement, vertically integrated platform stacks, and domestic-native systems—and adopts an analytical scaffold comprising a unified abstraction/observability base plus three main paths (execution and communication, state governance, compression-cost co-optimization) to compare where each route places its primary complexity. The survey distills structural challenges around hardware heterogeneity, complex workloads, unstable compression gains, and rapid upstream evolution, and outlines future directions including platform capability contracts, state-centric runtime restructuring, and unified control-plane governance.

Key Words domestic computing platforms; large-model inference; inference engine; runtime systems; hardware-software co-design

摘要 本文面向国产算力平台上的大模型推理引擎,围绕“生态格局—分析骨架—路线比较—关键挑战—未来收敛”这一主线展开结构化综述。以可交叉核验的公开证据为基础,本文首先梳理当前生态为何分化为开源主干复用增强、平台一体化方案与国产原生/独立路线三类,继而以“一个底座(统一抽象与可观测证据链)、三条主路径(执行与通信、状态治理、压缩与成本协同)”为分析骨架,比较不同路线把复杂度主要压在何处。在此基础上,将平台异构、复杂任务负载、压缩收益不稳定、上游快速演进与评测体系失真等问题收敛为结构性挑战,并讨论能力契约沉淀、状态中心化运行时重构和控制面一体化治理等未来方向,以期国产推理引擎的路线选择与架构演进提供可复核的分析框架。

关键词 国产算力平台; 大模型推理; 推理引擎; 运行时; 软硬协同

中图分类号: TP39